

Martin-Löf Type Theory

Part 2: Dependent types

Andreas Abel

DAT350 / DIT235 *Types for programs and proofs*
Chalmers University of Technology and University of Gothenburg

September 2026

Recapitulation

Four judgment forms:

- | | | | |
|---|------------------------------------|---|---------------------------|
| 1 | $\Gamma \vdash A \text{ type}$ | 3 | $\Gamma \vdash t : A$ |
| 2 | $\Gamma \vdash A = B \text{ type}$ | 4 | $\Gamma \vdash t = u : A$ |

Five principles per type former: formation, introduction, elimination, computation (β), extensionality (η).

In Part 1, type equality (2) was syntactic identity and the `CONV` rule was never used.

Part 2 allows types to contain terms.

Types that contain terms

```
module lecture4 where
open import Agda.Builtin.Nat renaming (Nat to  $\mathbb{N}$ )

data Vec (A : Set) :  $\mathbb{N}$  → Set where
  [] : Vec A zero
  _::_ : {n :  $\mathbb{N}$ } → A → Vec A n → Vec A (suc n)
infixr 5 _::_
```

$\text{Vec } A \ n$ is a type for every $n : \mathbb{N}$: type formation depends on terms.

The conversion rule, in use

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash A = B \text{ type}}{\Gamma \vdash t : B} \text{ CONV}$$

$\text{Vec } \mathbb{N} (2 + 2)$ and $\text{Vec } \mathbb{N} 4$ are distinct expressions. Yet, they are equal types, because $2 + 2 = 4 : \mathbb{N}$ by definition of $+$, and formation is a congruence.

$\text{ex} : \text{Vec } \mathbb{N} (2 + 2) \rightarrow \text{Vec } \mathbb{N} 4$
 $\text{ex } xs = xs$

Type equality (2) is now entangled with computation. Deciding it requires evaluation; this is why normalisation matters for type checking.

Dependent function type Π

$\Pi_{x:A}. B$: formation and introduction

1. Formation

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Pi_{x:A}. B \text{ type}}$$

2. Introduction

$$\frac{\Gamma, x:A \vdash t : B}{\Gamma \vdash \lambda x. t : \Pi_{x:A}. B}$$

The second premise of formation is a hypothetical judgment: B may mention x .

If x does not occur free in B , this is just function space $A \rightarrow B$.

$\Pi_{x:A}. B$: elimination, computation, extensionality

3. Elimination

$$\frac{\Gamma \vdash t : \Pi_{x:A}. B \quad \Gamma \vdash u : A}{\Gamma \vdash tu : B[u/x]}$$

The substitution in the *type* is the only new element for Π in comparison with $\rightarrow$. Otherwise, equality rules are unchanged.

$$\frac{\Gamma, x:A \vdash t : B \quad \Gamma \vdash u : A}{\Gamma \vdash (\lambda x. t) u = t[u/x] : B[u/x]} \quad \frac{\Gamma \vdash t : \Pi_{x:A}. B}{\Gamma \vdash t = \lambda x. tx : \Pi_{x:A}. B}$$

Curry–Howard: Π is universal quantification, $\rightarrow$ is implication.

Π in Agda

```
id : (A : Set) → A → A
```

```
id A x = x
```

```
replicate : {A : Set} (n : ℕ) → A → Vec A n
```

```
replicate zero a = []
```

```
replicate (suc n) a = a :: replicate n a
```

Agda writes $\Pi x:A. B$ as $(x : A) \rightarrow B$, and $\{x : A\} \rightarrow B$ for an argument to be inferred.

The types here quantify over `Set`. Which type do they inhabit? That question is answered at the end of this lecture.

Dependent sum Σ

$\Sigma_{x:A}. B$: formation and introduction

1. Formation

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma, x:A \vdash B \text{ type}}{\Gamma \vdash \Sigma_{x:A}. B \text{ type}}$$

2. Introduction

$$\frac{\Gamma \vdash t : A \quad \Gamma \vdash u : B[t/x]}{\Gamma \vdash \langle t, u \rangle : \Sigma_{x:A}. B}$$

The type of the second component depends on the first.

Curry–Howard: Σ is existential quantification, $\times$ is conjunction. An element is a witness together with a proof.

$\Sigma_{x:A}. B$: elimination, negative form

$$\frac{\Gamma \vdash t : \Sigma_{x:A}. B}{\Gamma \vdash \text{fst } t : A} \qquad \frac{\Gamma \vdash t : \Sigma_{x:A}. B}{\Gamma \vdash \text{snd } t : B[\text{fst } t/x]}$$

The type of the second projection mentions the first.

4. Computation $\Gamma \vdash \text{fst} \langle t, u \rangle = t : A$ and $\Gamma \vdash \text{snd} \langle t, u \rangle = u : B[t/x]$

5. Extensionality
$$\frac{\Gamma \vdash t : \Sigma_{x:A}. B}{\Gamma \vdash t = \langle \text{fst } t, \text{snd } t \rangle : \Sigma_{x:A}. B}$$

$\Sigma x:A. B$: elimination, positive form

With a motive C over the pair:

$$\frac{\Gamma, z:\Sigma x:A. B \vdash C \text{ type} \quad \Gamma \vdash t : \Sigma x:A. B \quad \Gamma, x:A, y:B \vdash e : C[\langle x, y \rangle / z]}{\Gamma \vdash \text{split } t (x.y.e) : C[t/z]} \text{ split}$$

4. Computation $\Gamma \vdash \text{split } \langle t, u \rangle (x.y.e) = e[t/x][u/y] : C[\langle t, u \rangle / z]$

The two presentations are interderivable, but only in the presence of the extensionality principle. Without η , projections are strictly weaker than split.

Σ in Agda

```
record  $\Sigma$  (A : Set) (B : A  $\rightarrow$  Set) : Set where
  constructor _,_
  field
    fst : A
    snd : B fst
open  $\Sigma$ 
```

The field `snd` has type `B fst`: a dependent field. Extensionality holds, as for `record` types in Part 1.

Σ in Agda: the positive presentation

```
module DataSigma where
  data  $\Sigma'$  (A : Set) (B : A  $\rightarrow$  Set) : Set where
     $\langle \_ , \_ \rangle$  : (a : A)  $\rightarrow$  B a  $\rightarrow$   $\Sigma'$  A B

  split : {A : Set} {B : A  $\rightarrow$  Set} (C :  $\Sigma'$  A B  $\rightarrow$  Set)
     $\rightarrow$  ((a : A) (b : B a)  $\rightarrow$  C  $\langle a , b \rangle$ )
     $\rightarrow$  (t :  $\Sigma'$  A B)  $\rightarrow$  C t
  split C e  $\langle a , b \rangle$  = e a b
```

The motive C makes `split` the dependent eliminator of §4. Extensionality does not hold for this presentation.

Dependent eliminators

From non-dependent to dependent elimination

In Part 1, every eliminator had a result type C that was fixed:

$$\frac{\Gamma \vdash b : \text{Bool} \quad \Gamma \vdash t : C \quad \Gamma \vdash f : C}{\Gamma \vdash \text{if } b \text{ then } t \text{ else } f : C}$$

Let C instead depend on the term being eliminated, as a hypothetical judgment $\Gamma, z:\text{Bool} \vdash C$ type. The branches are then checked at the corresponding instances of C , and the conclusion has type $C[b/z]$.

This is the only change. It is applied below to $+$, $\perp$ and $\mathbb{N}$.

$A + B$: from case analysis to a proof principle

$$\frac{\Gamma \vdash t : A + B \quad \Gamma, x_1:A \vdash t_1 : C \quad \Gamma, x_2:B \vdash t_2 : C}{\Gamma \vdash \text{case } t \text{ of } \lambda\{x_1 \rightarrow t_1; x_2 \rightarrow t_2\} : C} \text{ case}$$

Part 1: the result type C is fixed, and the rule is case analysis.

$A + B$: from case analysis to a proof principle

$$\frac{\Gamma, z:A + B \vdash C \text{ type} \quad \Gamma \vdash t : A + B \quad \Gamma, x_1:A \vdash t_1 : C[\text{in}_1 x_1/z] \quad \Gamma, x_2:B \vdash t_2 : C[\text{in}_2 x_2/z]}{\Gamma \vdash \text{case } t \text{ of } \lambda\{x_1 \rightarrow t_1; x_2 \rightarrow t_2\} : C[t/z]} \text{ case}$$

With a motive, the two branches prove C for the two possible forms of t , and the conclusion is C at t itself.

Curry–Howard: proof by cases on a disjunction.

Dependent elimination of $\perp$

$$\frac{\Gamma \vdash t : \perp}{\Gamma \vdash \text{absurd } t : C}$$

Part 1: $\text{absurd } t$ inhabits an arbitrary fixed type C .

Dependent elimination of $\perp$

$$\frac{\Gamma, z:\perp \vdash C \text{ type} \quad \Gamma \vdash t : \perp}{\Gamma \vdash \text{absurd } t : C[t/z]}$$

The rule barely changes: there are no branches, so there is nothing to check at an instance of the motive.

Curry–Howard: from a contradiction, any proposition follows — now including propositions that mention the contradiction.

$\mathbb{N}$: from recursion to induction

$$\frac{\Gamma \vdash c_z : C \quad \Gamma \vdash C \text{ type} \quad \Gamma, x:\mathbb{N}, y:C \vdash c_s : C \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ n : C} \text{ natrec}$$

Part 1: C is a fixed type, c_z its base value and c_s the step.

The computation principle is unchanged:

$$\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ \text{zero} = c_z : C[\text{zero}/x]$$

$$\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ (\text{suc } n) = c_s[n/x][\text{natrec } C \ c_z \ (x.y.c_s) \ n/y] : C[\text{suc } n/x]$$

$\mathbb{N}$: from recursion to induction

$$\frac{\Gamma \vdash c_z : C[\text{zero}/x] \quad \Gamma, x:\mathbb{N}, y:C \vdash c_s : C[\text{suc } x/x] \quad \Gamma \vdash n : \mathbb{N}}{\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ n : C[n/x]} \text{ natrec}$$

c_z proves C at zero; c_s proves C at $\text{suc } x$ given a proof y of C at x .

The computation principle is unchanged:

$$\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ \text{zero} = c_z : C[\text{zero}/x]$$

$$\Gamma \vdash \text{natrec } C \ c_z \ (x.y.c_s) \ (\text{suc } n) = c_s[n/x][\text{natrec } C \ c_z \ (x.y.c_s) \ n/y] : C[\text{suc } n/x]$$

Two readings of one rule

C constant

primitive recursion

c_z is a base value

c_s is the step function

Gödel's System T

C dependent

mathematical induction

c_z is the base case

c_s is the induction step

Peano arithmetic (FOL + induction)

The same rule, with the same computation principle.

A program and a proof by induction are the same object.

Dependent eliminators in Agda

`data _⊔_ (A B : Set) : Set where`

`inj1 : A → A ⊔ B`

`inj2 : B → A ⊔ B`

`elim⊔ : {A B : Set} (C : A ⊔ B → Set)`

`→ ((a : A) → C (inj1 a)) → ((b : B) → C (inj2 b))`

`→ (t : A ⊔ B) → C t`

`elim⊔ C f g (inj1 a) = f a`

`elim⊔ C f g (inj2 b) = g b`

`natrec : (C : ℕ → Set)`

`→ C zero → ((n : ℕ) → C n → C (suc n))`

`→ (n : ℕ) → C n`

`natrec C cz cs zero = cz`

`natrec C cz cs (suc n) = cs n (natrec C cz cs n)`

Exercise

Carry out the same step for Bool.

1. State the dependent elimination principle for Bool, with motive $\Gamma, z:\text{Bool} \vdash C$ type, and its two computation rules.
2. Use it to prove $\text{not}(\text{not } b) = b$. The motive $C := \text{Id}_{\text{Bool}}(\text{not}(\text{not } z), z)$ depends on z , so the eliminator of Part 1 does not suffice.
3. Carry out the proof in Agda twice: once through an explicit `elim-Bool`, once by pattern matching.

Part 2 §5 explains the identity type used in step 2.

The identity type

Why a type of equalities

Definitional equality, $\Gamma \vdash t = u : A$, is a *judgment*. It cannot be assumed, quantified over, negated, or proved by induction.

To reason about equality inside the theory, equality must be a *type*.

1. Formation

$$\frac{\Gamma \vdash A \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A}{\Gamma \vdash \text{Id}_A(a, b) \text{ type}}$$

2. Introduction

$$\frac{\Gamma \vdash a : A}{\Gamma \vdash \text{refl } a : \text{Id}_A(a, a)}$$

Elimination, restricted form: subst

3. Elimination (restricted to Leibniz' law)

$$\frac{\Gamma, y:A \vdash C \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma \vdash c : C[a/y]}{\Gamma \vdash \text{subst } C \ p \ c : C[b/y]} \text{ subst}$$

4. Computation $\Gamma \vdash \text{subst } C \ (\text{refl } a) \ c = c : C[a/y]$

Instantiating p to $\text{refl } a$ instantiates b to a .

Equals may be substituted for equals: Leibniz's principle, as an inference rule.

From subst one obtains sym, trans and cong. For instance
 $\text{sym } p := \text{subst } (y. \text{Id}_A(y, a)) \ p \ (\text{refl } a).$

subst in Agda

infix 4 $\equiv$

data $\equiv$ {A : Set} (x : A) : A → Set where

refl : x $\equiv$ x

subst : {A : Set} (C : A → Set) {a b : A} → a $\equiv$ b → C a → C b

subst C refl c = c

sym : {A : Set} {a b : A} → a $\equiv$ b → b $\equiv$ a

sym {A} {a} p = subst (λ y → y $\equiv$ a) p refl

trans : {A : Set} {a b c : A} → a $\equiv$ b → b $\equiv$ c → a $\equiv$ c

trans {A} {a} p q = subst (λ y → a $\equiv$ y) q p

cong : {A B : Set} (f : A → B) {a b : A} → a $\equiv$ b → f a $\equiv$ f b

cong f {a} p = subst (λ y → f a $\equiv$ f y) p refl

Where subst is needed

Append on vectors:

$_++_ : \{A : \text{Set}\} \{m\ n : \mathbb{N}\} \rightarrow \text{Vec } A\ m \rightarrow \text{Vec } A\ n \rightarrow \text{Vec } A\ (m + n)$

$[] ++\ ys = ys$

$(x :: xs) ++\ ys = x :: (xs ++\ ys)$

infixr 5 $_++_$

For $xs : \text{Vec } A\ n$, the expression $xs ++\ []$ has type $\text{Vec } A\ (n + \text{zero})$.

Is $n + \text{zero} = n : \mathbb{N}$?

The limits of conv

Addition recurses on its first argument:

$$\text{zero} + n = n \qquad \text{suc } m + n = \text{suc}(m + n)$$

$\text{zero} + n$ reduces to n .

$n + \text{zero}$ does not reduce: n is a variable, so neither computation rule applies.

$n + \text{zero} = n$ holds for every closed n , but it is not a judgmental equality: proving it requires induction on n . The CONV rule therefore does not apply.

CONV a rule; automatic; sees computation.

subst a term; supplied by the user; sees any proof.

The transport, explicitly

```
+{-identityr : (n : ℕ) → n + zero ≡ n
+{-identityr zero = refl
+{-identityr (suc n) = cong suc (+{-identityr n)
```

```
append-nil : {A : Set} {n : ℕ} → Vec A n → Vec A n
append-nil {A} {n} xs = subst (Vec A) (+{-identityr n) (xs ++ [])
```

The proof of $n + \text{zero} \equiv n$ is itself by induction — that is, by the dependent eliminator of §4, with motive $C := (\lambda z. \text{Id}_{\mathbb{N}}(z + \text{zero}, z))$.

Without the `subst`, Agda reports a mismatch between `Vec A (n + zero)` and `Vec A n`.

What subst cannot express

The motive of subst is $\Gamma, y:A \vdash C$ type. It mentions the *endpoint* y , but not the proof p .

So no statement *about a proof of equality* can be proved. For example, $\text{Id}_{\text{Id}_A(a,a)}(p, \text{refl } a)$ is outside its reach.

The general elimination principle generalises the motive to all three: the two endpoints and the proof.

From subst to J

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma, y:A \vdash C \text{ type} \quad \Gamma \vdash c : C[a]}{\Gamma \vdash \text{subst } C \, p \, c : C[b]} \text{ subst}$$

The motive has one hypothesis, and the base case is at the fixed endpoint a .

From subst to J

$$\frac{\Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma, x:A, y:A, q:\text{Id}_A(x, y) \vdash C \text{ type} \quad \Gamma, x:A \vdash c : C[x, x, \text{refl } x]}{\Gamma \vdash J \quad C p (x.c) : C[a, b, p]} J$$

The motive now has three hypotheses, and the base case must be proved for an arbitrary x at the proof $\text{refl } x$.

In words: to prove a statement about an arbitrary proof of equality, it suffices to prove it for refl .

J : computation, and subst recovered

4. Computation

$$\Gamma \vdash J\ C\ (\text{refl}\ a)\ (x.c) = c[a/x] : C[a, a, \text{refl}\ a]$$

subst is the instance of J whose motive ignores the proof:

$$\text{subst}\ C\ p\ c := J\ (x_1.x_2.q.\ C[x_1/y] \rightarrow C[x_2/y])\ p\ (x_1.\lambda w.w)\ c$$

Note what J does not say. The premise is that C holds at $\text{refl}\ x$ for arbitrary x ; it is not assumed that p is refl .

J in Agda

```
J : {A : Set} (C : (x y : A) → x ≡ y → Set)  
  → {a b : A} (p : a ≡ b)  
  → ((x : A) → C x x refl) → C a b p  
J C refl c = c _
```

```
subst' : {A : Set} (C : A → Set) {a b : A} → a ≡ b → C a → C b  
subst' C p = J (λ x y _ → C x → C y) p (λ x w → w)
```

Agda derives J by pattern matching on `refl`. Whether that is legitimate is the subject of the next section.

Extensionality for the identity type

What principle 5 would say

For a positive type, extensionality states that the eliminator is complete: every element is covered by the constructors.

$\text{Id}_A(a, b)$ has one constructor, refl. So the principle would say: *every proof of an equality is refl.*

This is **not derivable from J** .



Martin Hofmann (1965–2018) Thomas Streicher (1958–2025)

Hofmann and Streicher (1995) constructed a model interpreting identity types as **groupoids**.



Vladimir Voevodsky (1966–2017)

Voevodsky interprets types as spaces and identity types as homotopies, giving rise to **Homotopy Type Theory** (HoTT).

Axiom K

K adds the missing principle. Its motive fixes both endpoints to the same variable, and quantifies over a proof of a *loop*:

$$\frac{\Gamma \vdash a : A \quad \Gamma, x:A, q:\text{Id}_A(x, x) \vdash C \text{ type} \quad \Gamma \vdash q : \text{Id}_A(a, a) \quad \Gamma, x:A \vdash c : C[x, \text{refl } x]}{\Gamma \vdash K C q (x.c) : C[a, q]} K$$

Computation $\Gamma \vdash K C (\text{refl } a) (x.c) = c[a/x] : C[a, \text{refl } a]$

J motive over $x:A, y:A, q:\text{Id}_A(x, y)$
 K motive over $x:A, q:\text{Id}_A(x, x)$

K is equivalent to **UIP**: any two proofs of $\text{Id}_A(a, b)$ are equal.

Introduced by Thomas Streicher in his 1993 habilitation thesis.

K in Agda

```
K : {A : Set} (C : (x : A) → x ≡ x → Set)
  → {a : A} (q : a ≡ a)
  → ((x : A) → C x refl) → C a q
K C refl c = c _
```

```
uip : {A : Set} {a b : A} (p q : a ≡ b) → p ≡ q
uip refl refl = refl
```

Both are proved by pattern matching, and both are rejected under `--without-K`:

I'm not sure if there should be a case for the constructor `refl`, because I get stuck when trying to solve the following unification problems (inferred index $\neq$ expected index): $a \stackrel{?}{=} a$

Pattern matching on `refl` silently uses `K`. This is why eliminators and dependent pattern matching are equivalent only in the presence of `K`.

Three theories

Added	Consequence	Name
equality reflection $\frac{\Gamma \vdash p : \text{Id}_A(a, b)}{\Gamma \vdash a = b : A}$	judgment 4 absorbs Id; subst unnecessary; type checking undecidable	extensional TT
K / UIP	proofs of equality are unique; type checking decidable	intensional TT (Agda's default)
neither	$\text{Id}_A(a, b)$ carries structure; K is refutable	univalent foundations (HoTT)

These are three deliberate design choices, not one correct answer.

Extensionality, revisited

Two statements deferred from Part 1:

- The extensionality principle for $\mathbb{N}$ holds only *propositionally*. With the identity type it can be stated and, by induction, proved: any f satisfying the two computation rules is pointwise equal to natrec .
- *Function extensionality* — $(\prod x:A. \text{Id}_B(fx, gx)) \rightarrow \text{Id}_{A \rightarrow B}(f, g)$ — is a statement about Id , and is *not* derivable. It is independent of the η -rule for Π , which is a judgmental equality and does hold.

Alternative eliminators: J' and K'

J and K generalise *both* endpoints. They can also be formulated with the first endpoint held fixed in the motive.

$$\frac{\Gamma, y:A, q:\text{Id}_A(a, y) \vdash C \text{ type} \quad \Gamma \vdash a : A \quad \Gamma \vdash b : A \quad \Gamma \vdash p : \text{Id}_A(a, b) \quad \Gamma \vdash c : C[a, \text{refl } a]}{\Gamma \vdash J' C p c : C[b, p]} J'$$

$$\frac{\Gamma \vdash a : A \quad \Gamma, q:\text{Id}_A(a, a) \vdash C \text{ type} \quad \Gamma \vdash p : \text{Id}_A(a, a) \quad \Gamma \vdash c : C[\text{refl } a]}{\Gamma \vdash K' C p c : C[p]} K'$$

Since a is fixed, the base case no longer binds a variable: c replaces $(x.c)$.

Exercise: alternative eliminators are equivalent

Computation principles:

$$\Gamma \vdash J' \ C \ (\text{refl } a) \ c = c : C[a, \text{refl } a] \quad \Gamma \vdash K' \ C \ (\text{refl } a) \ c = c : C[\text{refl } a]$$

1. Define subst in terms of J' . Compare with the definition from J on §5: which motive does each require?
2. Define J' in terms of J , and J in terms of J' .
3. Define K' in terms of K , and K in terms of K' .
4. Carry out all of the above in Agda.

Hints (Claude): One direction of each of 2 and 3 is a matter of instantiating the motive. The other is not: the fixed endpoint a occurs in C , so it cannot be generalised by substitution. For 2, either quantify over the motive — which needs a universe (see later) — or use that every element of $\Sigma y:A. \text{Id}_A(a, y)$ is equal to $\langle a, \text{refl } a \rangle$. For 3, first prove $\prod p:\text{Id}_A(a, a). \text{Id}_{\text{Id}_A(a, a)}(p, \text{refl } a)$ from K .

Universes

Why a type of types

Throughout, $\Gamma \vdash A$ type has been a *judgment*. To quantify over types *inside* the theory, types must form a type.

Examples that require it:

- the polymorphic identity $\Pi A:\text{Type}. A \rightarrow A$;
- $\text{Vec} : \text{Type} \rightarrow \mathbb{N} \rightarrow \text{Type}$;
- any definition of a type by recursion on a natural number.

Two presentations: à la Russell and à la Tarski.

Universes à la Russell and à la Tarski

Russell. Elements of *Type* are types:

$$\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \text{Type type}} \qquad \frac{\Gamma \vdash A : \text{Type}}{\Gamma \vdash A \text{ type}}$$

Tarski. *Type* contains *codes*, decoded by *El*:

$$\frac{\Gamma \vdash a : \text{Type}}{\Gamma \vdash \text{El } a \text{ type}} \qquad \frac{\Gamma \vdash a : \text{Type} \quad \Gamma, x : \text{El } a \vdash b : \text{Type}}{\Gamma \vdash \hat{\Pi} a (x.b) : \text{Type}}$$

$$\Gamma \vdash \text{El}(\hat{\Pi} a (x.b)) = \Pi x : \text{El } a. \text{El } b \text{ type}$$

The Tarski presentation fits the five principles exactly: formation *Type*, introduction the codes, elimination *El*, computation the equations for *El*. Agda uses the Russell presentation.

Type : Type

The most convenient rule would be

$$\frac{\vdash \Gamma \text{ ctx}}{\Gamma \vdash \text{Type} : \text{Type}}$$

It is **inconsistent**. Girard (1972) derived a contradiction in System U; Hurkens (1995) gave a short version, about a page of λ -terms.

Consequences:

- there is a closed term of type $\perp$, so every type is inhabited and every proposition provable;
- normalisation fails: there is a term with no normal form.

This is Russell's paradox in type-theoretic form: the type of all types contains itself.

Type : Type in Agda

Agda provides the rule under a flag, for experimentation:

```
{-# OPTIONS --type-in-type #-}  
data ⊥ : Set where  
-- Hurkens' paradox, about 20 lines  
absurd : ⊥  
absurd = ...
```

The remedy: a predicative hierarchy.

$$\Gamma \vdash \text{Type}_i : \text{Type}_{i+1} \quad \frac{\Gamma \vdash A : \text{Type}_i \quad \Gamma, x:A \vdash B : \text{Type}_j}{\Gamma \vdash \Pi x:A. B : \text{Type}_{\max(i,j)}}$$

No universe contains itself. In Agda: `Set0`, `Set1`, ..., with universe polymorphism over levels.

Predicativity

$\text{poly} : (A : \text{Set}) \rightarrow A \rightarrow A$
 $\text{poly } A \ x = x$

The type of `poly` quantifies over `Set`, so it cannot itself belong to `Set`:

$_ : \text{Set}_1$
 $_ = (A : \text{Set}) \rightarrow A \rightarrow A$

A quantified type never ranges over a collection containing itself. This is what **predicativity** means, and the hierarchy enforces it.

Impredicativity: System F

System F (Girard 1972, Reynolds 1974) is *impredicative*: the type $\forall X. X \rightarrow X$ quantifies over all types, including itself.

Girard nevertheless proved System F strongly normalising, hence consistent.

However, impredicativity excludes important models of type theory, e.g., modeling types as sets.

Further, impredicativity is fragile: Adding a second impredicative universe on top is inconsistent, witnessed by Girard's paradox. In fact Hurken's paradox derives from it, the full power of $\text{Type} : \text{Type}$ is not needed.

Church encodings

In System F, data types can be encoded rather than declared:

$$\text{Nat} := \forall X. (X \rightarrow X) \rightarrow X \rightarrow X \quad \bar{n} := \Lambda X. \lambda s. \lambda z. s^n z$$

This yields the *simply typed* recursor of Part 1: iteration at any type.

It does **not** yield the dependent eliminator of §4. The induction principle for Church numerals is not derivable in System F (Geuvers 2001).

The dependent eliminator is genuinely stronger than the simply typed one.

Two designs

Martin-Löf type theory

predicative hierarchy of universes
inductive types with eliminators
induction available
large elimination available

System F, Prop

impredicative quantification
encodings
induction not derivable
elimination restricted

Both are consistent. They differ in what they make expressible.

Summary

The five principles, complete

	form.	intro	elim	comp	ext
$A \rightarrow B, \Pi x:A. B$	✓	λ	app	β	$t = \lambda x. t x$
$A \times B, \Sigma x:A. B$	✓	$\langle , \rangle$	fst, snd	2	$t = \langle \text{fst } t, \text{snd } t \rangle$
$\top$	✓	$\langle \rangle$	—	—	$t = \langle \rangle$
Bool, $A + B$	✓	2	case	2	$e[t/z] = \dots$
$\perp$	✓	—	absurd	—	only propositionally
$\mathbb{N}$	✓	zero, suc	natrec	2	only propositionally
$\text{Id}_A(a, b)$	✓	refl	J	1	absent ($K?$)
Type	✓	codes	El	1 per code	—

The last two rows are where the design decisions of the field are located.

Beyond this course

- W -types: a single former for all well-founded inductive types.
- Normalisation and decidability of judgments 2 and 4; the metatheory of the system presented here.
- Homotopy type theory: Id as a path type, univalence, higher inductive types. Cubical Agda.
- Implementations: Agda, Rocq, Lean, Idris.

References

- Per Martin-Löf. *Intuitionistic Type Theory*. Bibliopolis, 1984.
- Bengt Nordström, Kent Petersson, Jan M. Smith. *Programming in Martin-Löf's Type Theory*. Oxford, 1990.
- Jean-Yves Girard. *Interprétation fonctionnelle et élimination des coupures*. Thèse, Paris VII, 1972.
(System F; the paradox)
- Antonius J.C. Hurkens. A simplification of Girard's paradox. *TLCA*, 1995.
- Herman Geuvers. Induction is not derivable in second order dependent type theory. *TLCA*, 2001.
- Thomas Streicher. *Investigations into Intensional Type Theory*. Habilitation, München, 1993.
(axiom K)
- Martin Hofmann, Thomas Streicher. The groupoid interpretation of type theory. In *Twenty-Five Years of Constructive Type Theory*, Oxford, 1998.
(refutes UIP)
- Jesper Cockx, Dominique Devriese, Frank Piessens. Pattern matching without K. *ICFP*, 2014.
- Vladimir Voevodsky. A very short note on homotopy λ -calculus. Unpublished note, 2006.
(univalence)
- The Univalent Foundations Program. *Homotopy Type Theory*. Institute for Advanced Study, 2013.